gRPC

HTTP/2 Protocol Buffers Streaming



01

Contexto e historia

De dónde viene gRPC, qué problema resuelve y cómo viaja una llamada de un servicio a otro.

De Stubby a gRPC

HTTP/2

Protobuf

gRPC · Remote Procedure Call

Framework multiplataforma y de código abierto, creado por Google para conectar servicios entre sí a máxima velocidad.

Multiplataforma | Código abierto | Contrato .proto

CLIENTE · NODE.JS

```
const producto =
  await getProductById(client, "2");
```

viaja por HTTP/2 como bytes Protobuf

SERVIDOR · .NET (C#)

```
→ Product {
  id: "2", name: "Papel",
  quantity: 20 }
```

HISTORIA

~2001

Stubby

RPC interno de Google para conectar sus microservicios.

Mar 2015

Código abierto

Google libera la siguiente versión de Stubby, ya sobre HTTP/2.

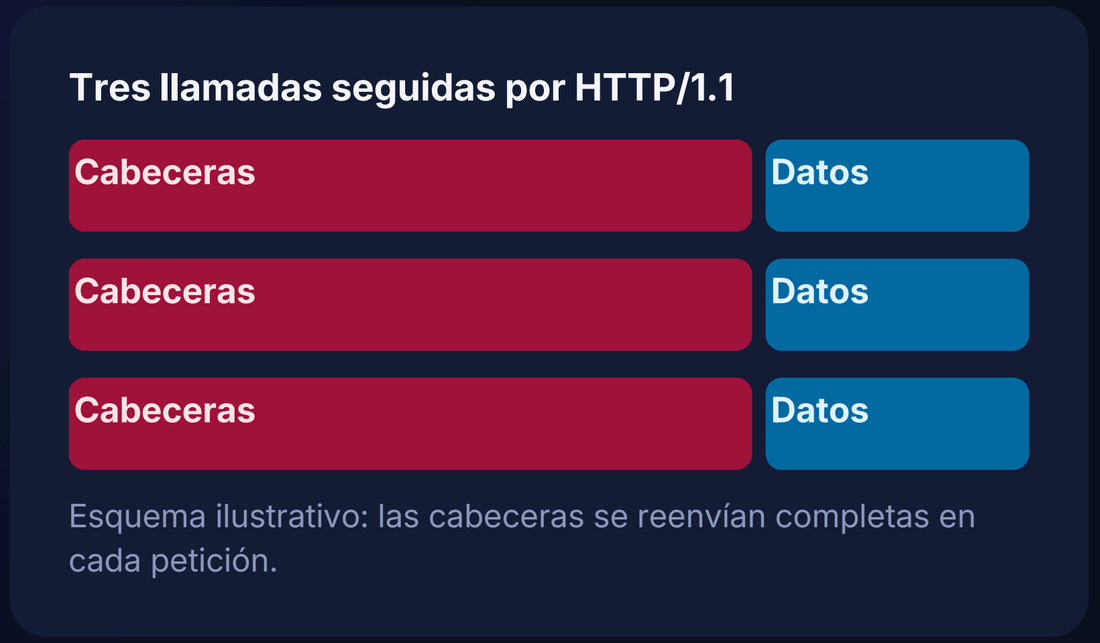
Ago 2016

gRPC 1.0 estable y lista para producción en varios lenguajes.

El problema

Microservicios masivos saturados por cabeceras HTTP repetidas.

- JSON en texto plano: más bytes y más CPU
- Peticiones que esperan su turno en HTTP/1.1



Esquema ilustrativo: las cabeceras se reenvían completas en cada petición.

LA RESPUESTA DE GRPC

Contratos fuertes

Un archivo .proto define métodos y mensajes con tipos estrictos. Cliente y servidor lo comparten.

Multiplexación

HTTP/2 lleva muchas llamadas en paralelo por una sola conexión y comprime las cabeceras.

Interoperabilidad

Del mismo contrato sale código para 11 lenguajes: un servidor C# atiende a un cliente Node.js.

ARQUITECTURA Y FUNCIONAMIENTO

Cómo viaja una llamada

CLIENTE

código

Llama a un método como si fuera local.

CLIENTE

Stub

Serializa el mensaje a bytes Protobuf.

RED

HTTP/2

Un canal con muchas llamadas y cabeceras comprimidas.

SERVIDOR

Código generado

Deserializa y despacha al método correcto.

SERVIDOR

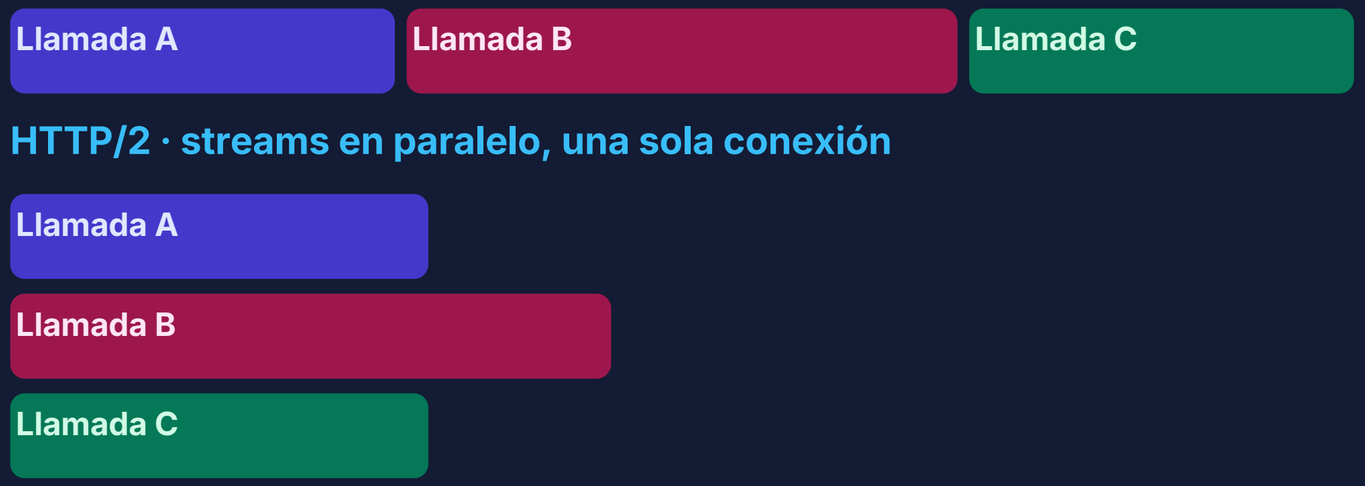
lógica

WarehouseServic e busca el producto y responde.

La respuesta regresa por el mismo camino, en sentido inverso.

TRANSPORTE

HTTP/2 por debajo



Multiplexación
Muchas llamadas en paralelo sobre una sola conexión.

Compresión HPACK
Las cabeceras se comprimen y no se reenvían completas.

Streams bidireccionales
La base técnica del streaming de gRPC.

FORMATO DE LOS DATOS

JSON

```
{"id":"1","name":"Queso","quantity":10}
```

39 bytes

- Texto plano con los nombres de campo en cada mensaje
- Sin esquema estricto

Protobuf

```
0A 01 31 12 05 51 75 65 73 6F 18 0A
```

12 bytes

- Binario: etiqueta numérica de campo y valor
- Esquema obligatorio y fuertemente tipado

Mensaje Product del repositorio: id "1", name "Queso", quantity 10. JSON compacto, sin espacios. Los bytes en rosa son las etiquetas de campo 1, 2 y 3.

VOCABULARIO

RPC
Llamada a Procedimiento Remoto. Las llamadas remotas parecen y funcionan como llamadas locales.

Stub
Componente generado en el cliente. Implementa los mismos métodos del servicio y envía la solicitud al servidor.

.proto
Archivo de contrato. Define los servicios y los mensajes estructurados.

Message
Modelo de datos del contrato. Cada campo lleva tipo y número: int32 quantity = 3;

Service
Agrupa los métodos remotos que expone el servidor, como un controlador.

Serialización
Conversión de objetos a bytes antes de viajar por la red.

Datos, características y usos

02

El contrato, los cuatro modos de comunicación, las ventajas, los límites y la comparación con REST.

Contrato .proto

Streaming

gRPC vs REST

VOCABULARIO · BLOQUE 2

Vocabulario clave

Protobuf

El formato de datos de gRPC. Serializa la información en binario para que viaje más rápido.

Streaming

Conexión abierta en HTTP/2 para un flujo continuo de mensajes: un chorro de agua, no gotas aisladas.

HTTP/2

La autopista por donde viaja gRPC. Hace posibles el streaming y la alta velocidad.

Contract-First

Diseñar primero el .proto, un IDL. El código sale de él: protoc en C#, proto-loader en Node.

Deadline

Tiempo máximo que espera el cliente. Al vencer, la llamada termina con DEADLINE_EXCEEDED.

Interceptor

Código que envuelve cada llamada para registrar, medir o manejar errores, como en la demo.

EL CONTRATO

Definiendo el almacén

service Warehouse

Agrupa los cuatro métodos remotos que expone el servidor.

message Product

Define la forma de los datos: id, nombre y cantidad.

= 1; = 2; = 3;

Números de campo que viajan por la red. No se cambian una vez publicada la API.

experimento.proto

```
syntax = "proto3";
package warehouse;

service Warehouse {
  rpc GetProductById (ProductID) returns (Product);
  rpc GetProductByName (ProductName) returns (stream Product);
  rpc AddProduct (Product) returns (ProductID);
  rpc UpdateProduct (Product) returns (Product);
}

message Product {
  string id = 1;
  string name = 2;
  int32 quantity = 3;
}
message ProductID { string id = 1; }
message ProductName { string name = 1; }
```

Extracto de GrpcServer/Protos/experimento.proto del repositorio

CÓMO FLUYEN LOS DATOS

Cuatro modos de comunicación

MODO 1

Unaria

Cliente envía



Servidor responde



```
(Req) returns (Res)
```

Una petición, una respuesta. Como llamar a una función.

En la demo: AddProduct, GetProductById, UpdateProduct

MODO 2

Server streaming

Cliente envía



Servidor responde



```
(Req) returns (stream Res)
```

Una petición y un flujo de respuestas ordenadas.

En la demo: GetProductByName

MODO 3

Client streaming

Cliente envía



Servidor responde



```
(stream Req) returns (Res)
```

Un flujo de peticiones y una respuesta al final.

No se usa en la demo

MODO 4

Bidireccional

Cliente envía



Servidor responde



```
(stream Req) returns (stream Res)
```

Dos flujos independientes que leen y escriben a su

No se usa en la demo

MANEJO DE ERRORES

Códigos de estado: gRPC frente a HTTP

Código gRPC	Número	Equivalente HTTP	Cuándo se usa
OK	0	200 OK	Operación completada con éxito
InvalidArgument	3	400 Bad Request	Faltan datos requeridos o son inválidos
NotFound	5	404 Not Found	El ID o recurso solicitado no existe
AlreadyExists	6	409 Conflict	Ya existe un registro con ese identificador
Internal	13	500 Internal Error	Error no controlado en el servidor

NotFound al buscar el producto con ID 99

AlreadyExists al agregar de nuevo el ID 1

InvalidArgument si el producto llega sin ID

CARACTERÍSTICAS

Más allá de la llamada

Deadlines

El cliente fija cuánto espera. Si se pasa, la llamada termina con DEADLINE_EXCEEDED.

Cancelación

Cliente o servidor pueden cancelar en cualquier momento. Lo ya hecho no se revierte.

Metadata

Pares clave-valor por llamada, por ejemplo las credenciales de autenticación.

Interceptores

Código que envuelve cada llamada para registrar, medir o autenticar.

Reintentos y salud

Políticas de reintento y chequeos de salud para balancear carga.

Reflexión

El servidor describe sus servicios y herramientas como Postman los descubren.

SEGURIDAD

Dos puertos, dos propósitos

DESARROLLO LOCAL

5230

HTTP/2 sin TLS (h2c)

Lo usa el cliente Node.js con credenciales inseguras: createInsecure().

Sin cifrar: solo para

CON TLS

7019

HTTP/2 con TLS

Lo usan Postman y grpcurl. El certificado de desarrollo es autofirmado.

Cifrado con TLS

En producción, siempre TLS. La autenticación viaja como metadata de cada llamada.

FORTALEZAS

Ventajas críticas

Alto rendimiento

Protobuf binario y HTTP/2 multiplexado: menos bytes y menos esperas.

En la demo: 12 bytes frente a 39 en JSON

Contratos estrictos

El .proto es obligatorio y tipado. Cliente y servidor hablan el mismo idioma.

En la demo: Product con id, name y quantity de tipo fijo

Autogeneración

Del .proto salen la base del servidor y el cliente listos para usar.

En la demo: Warehouse.WarehouseBase en C# y el stub en Node.js

Streaming nativo

Flujos de peticiones y respuestas sobre HTTP/2, sin recurrir a WebSockets.

En la demo: GetProductByName entrega cada producto al

LÍMITES

Retos y limitaciones

Tráfico opaco

Los mensajes binarios no se leen a simple vista.

CÓMO SE MITIGA

Reflexión del servidor, Postman y grpcurl. Protobuf se puede convertir a JSON para depurar.

Navegadores

Un navegador no controla HTTP/2 lo bastante para llamar gRPC directo.

CÓMO SE MITIGA

gRPC-Web con un proxy, o transcodificación JSON para exponer una API REST.

Curva de aprendizaje

Hay que aprender .proto, generación de código y HTTP/2.

CÓMO SE MITIGA

Diseño contract-first: un solo archivo guía al cliente y al servidor.

Sin difusión

No existe enviar un mensaje a todos los clientes registrados: cada llamada hace su propio stream.

CÓMO SE MITIGA

Para difusión en tiempo real, SignalR es una alternativa.

COMPARACIÓN

gRPC frente a REST

Dimensión	gRPC	REST (HTTP y JSON)
Contrato	Obligatorio (.proto)	Opcional (OpenAPI)
Formato	Binario (Protobuf)	Texto (JSON)
Protocolo	HTTP/2	HTTP, generalmente 1.1
Modelo	Operaciones (RPC)	Recursos y verbos HTTP
Tipado	Fuerte, en compilación	Débil o dinámico
Streaming	Cliente, servidor y bidireccional	Limitado (SSE o WebSockets)
Navegador	Requiere gRPC-Web	Soporte directo
Cliente	Generado desde el .proto	OpenAPI y herramientas externas

REST también puede usar HTTP/2, así que la diferencia no depende solo del transporte.

03

Aplicaciones y demostración

Casos reales

Cuándo usarlo

Demo funcional

APLICACIONES REALES

Casos de uso

Square

Sustituyó su RPC propio por gRPC: multiplataforma, buen rendimiento y adaptable a su red.

Netflix

Lo adoptó para extender su ecosistema y desarrollar también en lenguajes distintos de la JVM.

Cockroach Labs

Migró de un RPC propio sin fricción y aprovechó el control de flujo por stream.

Cisco

Lo usa como transporte unificado para configuración y telemetría, con streaming bidireccional y TLS.

Uber

Su plataforma de mensajes en tiempo real usa streaming bidireccional gRPC en las apps de pasajeros, conductores y Eats.

Dropbox

En 2019 anunció que la siguiente versión de su framework Courier se apoyaría en gRPC.

Fuentes: grpc.io/about, Wikipedia (Dropbox) y Uber Engineering (2022)

CUÁNDO CONVIENE OTRA OPCIÓN

Escenarios no recomendados

APIs accesibles desde el navegador

gRPC no está soportado por completo en el navegador. gRPC-Web ayuda, pero añade un proxy y tiene límites.

ALTERNATIVA

REST con JSON, o gRPC-Web

Difusión a muchos clientes

No existe el envío a todos los clientes registrados: cada llamada hace su propio stream, como en un chat.

ALTERNATIVA

SignalR

Mensajes legibles por humanos

Protobuf es binario y exige el .proto y herramientas para leerlo o escribir peticiones a mano.

ALTERNATIVA

REST con JSON, o grpcurl y Postman

Fuente: Microsoft Learn (comparación de servicios gRPC con APIs HTTP)

DEMOSTRACIÓN FUNCIONAL

Almacén gRPC con C# y Node.js

github.com/Lyttchoide/grpc-warehouse




```
grpc-warehouse/
├── GrpcServer/
│   ├── Protos/experimento.proto
│   ├── Services/WarehouseService.cs
│   ├── ErrorHandllingIterceptor.cs
│   ├── Program.cs
│   └── appsettings.json
├── NodeClient/
│   ├── protos/experimento.proto
│   ├── warehouseClient.js
│   └── client.js
└── .example.env
```

IMPLEMENTACIÓN · SERVIDOR C# (.NET)

WarehouseService

UNARIO · GETPRODUCTBYID

```
public override Task<Product> GetProductById(
  ProductID request, ServerCallContext context)
{
  if (_products.TryGetValue(request.Id, out var p))
    return Task.FromResult(p);

  throw new RpcException(new Status(
    StatusCode.NotFound, "No se encontró"));
}
```

Si el ID no existe, lanza una RpcException con StatusCode.NotFound.

SERVER STREAMING · GETPRODUCTBYNAME

```
public override async Task GetProductByName(
  ProductName request,
  IServerStreamWriter<Product> responseStream,
  ServerCallContext context)
{
  foreach (var product in matchingProducts)
  {
    await responseStream.WriteAsync(product);
    await Task.Delay(200);
  }
}
```

Cada coincidencia se escribe en el canal abierto, con una pausa de 200 ms.

Extracto simplificado de GrpcServer/Services/WarehouseService.cs

IMPLEMENTACIÓN · SERVIDOR C# (.NET)

ErrorHandlingInterceptor

INTERCEPTOR UNARIO

```
public override async Task<TResponse> UnaryServerHandler(
    TRequest request, ServerCallContext context,
    UnaryServerMethod<TRequest, TResponse> continuation)
{
  try { return await continuation(request, context); }
  catch (RpcException rpcEx) {
    Console.WriteLine($"gRPC: {rpcEx.Status.Detail}");
    throw; // se propaga al cliente
  }
  catch (Exception ex) {
    Console.WriteLine($"General: {ex.Message}");
    throw new RpcException(new Status(
      StatusCode.Internal, "Ocurrió un error..."));
  }
}
```

Error de gRPC: se relanza

Se registra en el servidor y el cliente recibe el código original: 3, 5 o 6.

Cualquier otro error: Internal

El detalle queda en el servidor y el cliente solo ve un mensaje genérico, código 13.

Solo llamadas unarias

GetProductByName es streaming y no pasa por este interceptor.

Extracto simplificado de GrpcServer/ErrorHandlingInterceptor.cs · se activa con AddGrpc en Program.cs

INVOCACIÓN · CLIENTE NODE.JS

Llamar al servidor

WAREHOUSECLIENT.JS · STREAM

```
export function getProductByName(client, name) {
  return new Promise((resolve, reject) => {
    const products = [];
    const call = client.GetProductByName({ name });
    call.on("data", (p) => products.push(p));
    call.on("end", () => resolve(products));
    call.on("error", reject);
  });
}
```

El stream emite un evento data por cada producto y end al terminar.

CLIENT.JS · USO

```
const client = createClient(SERVER_ADDRESS);

try {
  const prod = await getProductById(client, "2");
  console.log(`Encontrado: ${prod.name}`);
} catch (err) {
  console.error(`[${err.code}] ${err.details}`);
}

const r = await getProductByName(client, "Papel");
```

Los callbacks de Node.js se envuelven en promesas para usar async/await.

Extracto simplificado de NodeClient/warehouseClient.js y client.js

RESULTADO

Ejecución y salida del cliente

TERMINAL 1 · SERVIDOR

```
$ cd GrpcServer
$ dotnet run
```

TERMINAL 2 · CLIENTE

```
$ cd NodeClient
$ npm start
```

Antes: copiar .example.env a .env (también en NodeClient). Requiere .NET SDK 7.0 y Node.js 18.

```
─── 1. AddProduct ───
Producto agregado — ID: 1, Nombre: Queso, Cantidad: 10
Error esperado al agregar 'Queso Duplicado': [6] El producto con ID '1'
ya existe en el almacén.
─── 2. GetProductById ───
Producto encontrado — ID: 2, Nombre: Papel, Cantidad: 20
Error al obtener ID '5': [5] No se encontró el producto con ID: 5
─── 3. UpdateProduct ───
Producto actualizado — ID: 1, Nombre: Queso Gouda, Cantidad: 50
─── 4. GetProductByName ───
Buscando productos con nombre: "Papel"
[Stream] Recibido: Papel (Cantidad: 20)
Total recibidos via streaming: 1
Error en streaming para 'NoExiste': [5] No se encontraron productos con
el nombre: NoExiste
```

Salida de client.js. Si lo repites sin reiniciar el servidor, los AddProduct dan [6].

CONCLUSIÓN

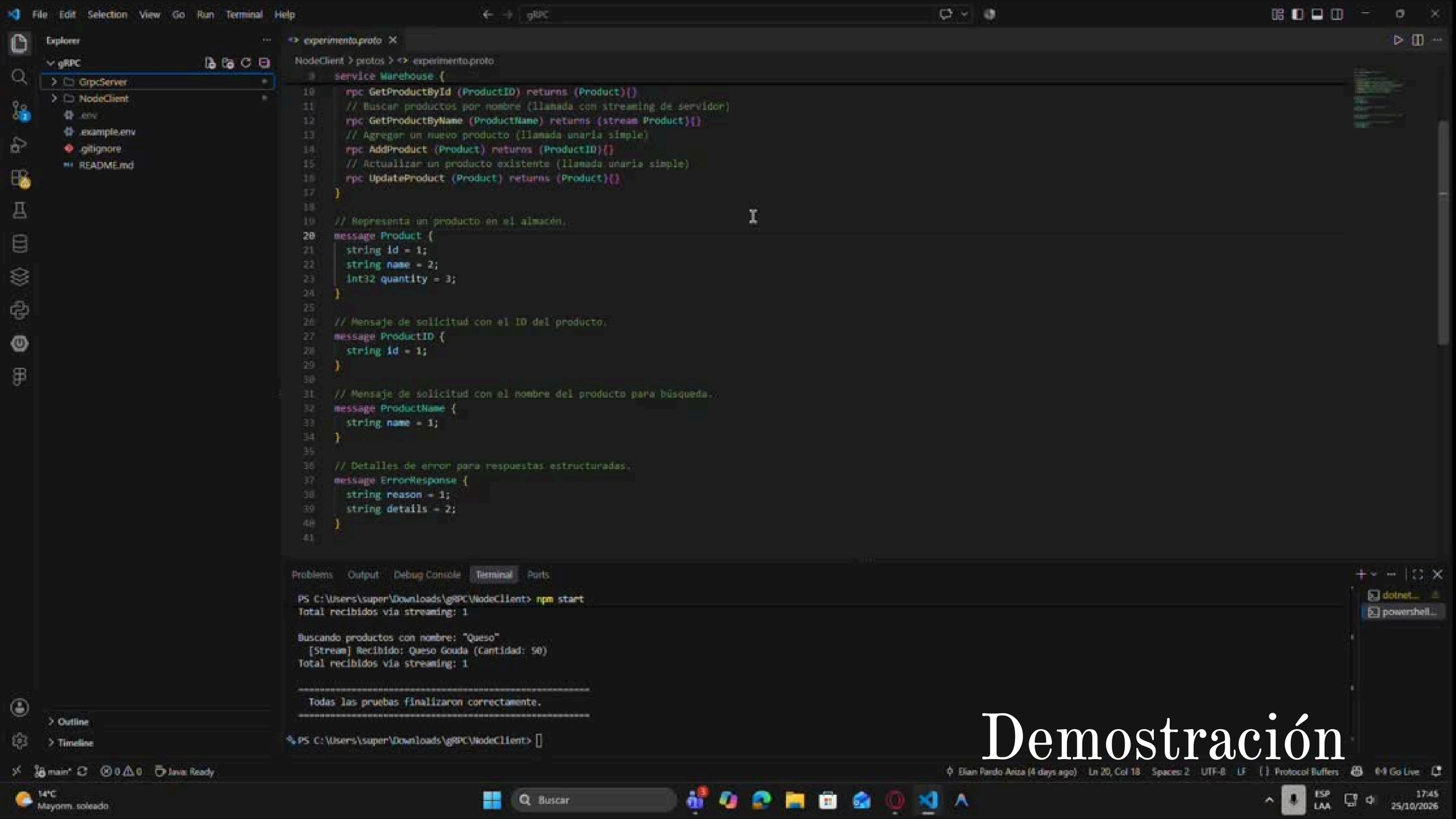
gRPC es la herramienta correcta para la comunicación entre

Rápido y compacto
HTTP/2 y Protobuf reducen bytes y esperas.

Contrato primero
Un solo .proto alinea cliente y servidor en cualquier lenguaje.

Con sus límites
Para navegadores y difusión, REST o SignalR siguen siendo mejores.

> En nuestra demo, un servidor C# y un cliente Node.js se entienden gracias a un único archivo: experimento.proto.



Demostración

REFERENCIAS

Fuentes y enlaces

Sobre gRPC	https://grpc.io/about/
Conceptos básicos	https://grpc.io/docs/what-is-grpc/core-concepts/
Anuncio de gRPC 1.0	https://grpc.io/blog/gablogpost
gRPC en la CNCF	https://www.cncf.io/projects/grpc/
gRPC frente a APIs HTTP	https://learn.microsoft.com/en-us/aspnet/core/grpc/comparison
Historia y adoptantes	https://en.wikipedia.org/wiki/GRPC
Uber y gRPC (2022)	https://www.uber.com/blog/ubers-next-gen-push-platform-on-grpc/
Código de la demo	https://github.com/Lyttchoide/grpc-warehouse

Los 12 y 39 bytes son del mensaje de ejemplo de la demo, no un benchmark general.